

Le hub de preparation Nutrition

Etape 4 du chantier Nutrition — ft-v1204 — 13/09/2026. Dix-septieme document de la serie, premier de l'etape 4. Tous les decomptes de ce document sont **recomptes depuis le code servi** a chaque generation : 21 gardes refusent de produire le PDF si un seul fait tombe.

LA BORNE POSEE PAR MICHEL, ET ELLE DECIDE DE TOUT

« Je ne veux **PAS** encore la douane. Le hub doit d'abord etre construit et valide seul. » — et le cahier des charges est ecrit **en negatif** : ne decide pas si une ligne est valide · ne corrige aucune valeur · ne bloque aucun enregistrement · ne modifie ni quantite, ni unite, ni pour-100 g, ni provenance, ni portion · ne touche pas encore a S. foodLog.

Objectif, mot pour mot : *faire converger les portes d'entree Nutrition vers un chemin commun de preparation d'un aliment, sans modifier ce qui est enregistre aujourd'hui.*

1. Les portes reelles — 12, cartographiees AVANT toute ligne

12 fonctions appellent `_afOublierAliment` (la remise a zero d'un aliment). Classees par **ROLE**, jamais par ressemblance.

groupe	fonctions	etat
A — passent deja par <code>_offRemplirFormulaire</code>	<code>_lookupBarcode</code> · <code>_calAppliquer</code> · <code>_afSuggPrendreMarque</code> · <code>_afSuggPrendreCiqual</code> · <code>_afSuggPrendreOff</code>	5 — migrees d'un coup, par leur hub
B — preparent l'ecran a la main	<code>quickFillFood</code> · <code>_afSuggPrendreLocale</code> · <code>onFoodLabelFile</code> · <code>estimateFoodAI</code>	4 — une seule migree
C — ne preparent pas un aliment	<code>openAddFood</code> (l'ouverture) · <code>_bcSansValeurs</code> (repli « sans valeurs ») · <code>quickAddFood</code> (ajout direct , sans ecran)	3 — hors sujet par nature

ET LE PREMIER RELEVÉ ÉTAIT FAUX — BUGS.md §63, DANS MON PROPRE INSTRUMENT

Il attribuait des appels a `readFoodLabel` et `scanBarcodeIA`. **Ce ne sont PAS des portes** : sept lignes chacune, elles n'ouvrent que le selecteur de fichier. Les vraies sont `onFoodLabelFile` et `estimateFoodAI`.

Cause : l'extracteur bornait les corps par `\nfunction NOM(` et **ratait les** `async function`. *Un motif qui suppose une syntaxe ne compte pas les endroits : il compte les endroits ecrits comme on les imaginait.*

Ce qui l'a attrape n'est pas une relecture, mais **une ligne de points qui ne pouvait pas etre vide** — une porte sans un seul geste d'ecran.

2. Le noyau commun — mesure geste par geste, pas estime

`onFoodLabelFile` recopiait **8 des 13 gestes** de `_offRemplirFormulaire`, et **son propre commentaire le disait depuis ft-v1163** : « *c'est exactement ce que faisait _offRemplirFormulaire* ». Mais elle en **OMET 5** — donc le hub ne prend que les 8.

geste	<code>_offRemplirFormulaire</code>	<code>onFoodLabelFile</code>	dans le hub ?
vider le champ des grammes	oui	oui	OUI
<code>_bcQtyPose = false</code>	oui	oui	OUI
proposer la portion	oui	oui	OUI
ecrire d'ou vient ce nombre	oui	oui	OUI
poser le nom	oui	oui	OUI
montrer la ligne	oui	oui	OUI
poser la description	oui	oui	OUI
recalculer	oui	oui	OUI

geste	_offRemplirFormulaire	onFoodLabelFile	dans le hub ?
pastille « la dernière fois »	oui	NON	non
le paquet	oui	NON	non
la provenance (_afSetSrc)	oui	oui (<i>la sienne</i>)	non
l'état (_afNoteEtat)	oui	NON	non
la carte sante	oui	NON	non

LA PHRASE QUI A DECIDE DU PERIMETRE

Un hub qui fait plus que le noyau commun n'est pas un chemin commun : c'est une porte qui en avale une autre.
 Absorber les 5 gestes manquants changerait le comportement de onFoodLabelFile — **mesure, pas suppose** : la mutation « je finis le hub, j'y mets la pastille » fait rougir exactement le témoin de ces 5 gestes omis.

Le hub, tel qu'il est dans app.js aujourd'hui — extrait **en direct** du code servi, commentaires de fin de ligne retirés :

```
function _afPreparerEcran(nut, portion, etiquettePortion, sourcePortion, suffixeNom){
  const g = parseFloat(portion) > 0 ? parseFloat(portion) : 0;
  const gramsEl = document.getElementById('af-bc-grams'); if(gramsEl) gramsEl.value = '';
  _bcQtyPose = false;
  _bcProposerPortion(g, etiquettePortion);
  _bcQsrc(g, sourcePortion);
  const nameEl = document.getElementById('af-bc-name');
  if(nameEl) nameEl.textContent = nut.name + ' . ' + nut.kcal100 + ' kcal/100g' + (suffixeNom || '');
  const row = document.getElementById('af-bc-row'); if(row) row.style.display = 'block';
  const descEl = document.getElementById('af-desc'); if(descEl) descEl.value = nut.name;
  _bcApplyGrams();
}
```

ET LES TROIS LIBELLES RESTENT DES PARAMETRES, PAS UNE REGLE

Chaque porte sait *d'où vient* le nombre qu'elle propose : « portion fabricant », « lu sur l'étiquette », ou **l'enseigne** pour un produit de marque. Le hub, lui, ne le sait pas — **une formule unique lui ferait inventer une source**. *C'est une DONNEE propre à la porte, pas une règle à factoriser* (R29 : on ne devine pas ce qu'on n'a pas lu).

3. Ce que le hub NE fait pas — et c'est la moitié de sa définition

Le hub **n'est PAS la douane**. Il **prepare** ; la douane validera plus tard. Chaque refus est **figé par un témoin**, et quand c'est possible par **deux** — un témoin de comportement (on pose un état, on appelle le hub, on relit) et un témoin de source.

il ne...	fige par
ne juge pas si une ligne est valide	témoin de source (aucun toast, aucune validation)
ne corrige aucune valeur	même témoin de source
ne bloque aucun enregistrement	même témoin de source
ne touche pas à S. foodLog	comportement ET source
ne touche pas à la provenance (<code>_afSetSrc</code>)	comportement + source
ne touche ni quantité , ni unité , ni pour-100 g , ni portion	comportement + source
ne touche aucun propriétaire de 1b/3 (<code>_qGrammes</code> , <code>_afReprendre...</code> , <code>_srcProvenance...</code>)	témoin de source dédiée

POURQUOI UN TÉMOIN DE SOURCE, ALORS QUE LES COMPORTEMENTS SONT VERTS

Parce qu'un hub qui déborde **reste vert à l'écran**. Le jour où quelqu'un ajoutera une validation dans le hub « parce que c'est l'endroit logique », aucun parcours ne changera de résultat — l'écran affichera la même chose. C'est la leçon de ft-v1202, redite ici sur un objet plus gros : **une dérive de conception peut être invisible à l'exécution**.

4. Non-fusion par ressemblance — la question posée nommément par Michel

« *vérifie qu'aucune porte n'est fusionnée simplement parce qu'elle "ressemble" à une autre* ». **Mesure** : part des 8 gestes du noyau que chaque porte restée dehors fait déjà en propre.

porte	gestes du noyau	pourquoi elle ne migre pas
<code>quickFillFood</code>	4 / 8	ni portion proposée, ni recalcul, ni description -> l'y brancher AJOUTERAIT des gestes
<code>_afSuggPrendreLocale</code>	5 / 8	idem (ni portion proposée, ni recalcul)
<code>_bcSansValeurs</code>	2 / 8	c'est un repli , pas un aliment préparé : elle ouvre la saisie manuelle
<code>estimateFoodAI</code>	1 / 8	elle ne prépare pas l'écran, elle pose une estimation
<code>openAddFood</code>	1 / 8	c'est l'ouverture de l'écran, pas une porte d'aliment
<code>quickAddFood</code>	0 / 8	aucun écran : elle écrit directement

Deux témoins figent cette non-fusion, et la mutation « *quickFillFood ressemble, je la branche aussi* » fait **2 rouges**. La ressemblance n'est pas un métier — c'est la règle qui a écarté deux sous-étapes entières du chantier 1b/3.

5. La sonde — elle conduisait, elle n'observait pas l'écran

Les **23** cles existantes observent `_afSrc`, la pastille, le trio des grammes, la définition de portion — **jamais** le champ des grammes, la portion proposée, le nom, la ligne visible ni la description, qui sont *exactement* ce que le hub déplace. Un instantané serait donc resté identique **quoi qu'on fasse au hub**. Étendue à **25 cles AVANT le BEFORE**, jamais après.

ET ELLE SALIT L'ECRAN AVANT CHAQUE CAS

Elle écrit une valeur témoin dans **tous** les champs avant d'appeler le chemin mesure. Sans ça, un champ **déjà vide** serait indiscernable d'un champ **vide par le hub** — et le témoin « le hub vide le champ des grammes » serait un vert qui ne peut pas rougir.

UN EXTRACTEUR DE TEMOIN RENDAIT DU VIDE : DEUX TEMOINS PASSAIENT A VIDE

La regex était écrite '\\\\(' cote source, soit **un antislash suivi d'une ouverture de groupe** au lieu d'une parenthèse littérale : elle ne trouvait **aucun** corps de fonction. Or deux des témoins testent une **ABSENCE** — et une absence est toujours vraie dans une chaîne vide. Ils étaient **verts sans rien mesurer**.

C'est le **seul témoin qui teste une PRESENCE** qui les a trahis, en rougissant sur du code parfaitement sain. *Un témoin de source se vérifie d'abord contre le code SAIN (ft-v1198)*. L'extracteur **LEVE** désormais au lieu de rendre du vide : l'erreur est impossible à refaire.

ET J'AI CORRIGE CETTE ANALYSE DANS LE MAUVAIS SENS AVANT DE LA CORRIGER DANS LE BON

Mon premier diagnostic était juste ; un test lancé en ligne de commande m'a fait dire « *je me suis trompé, l'extracteur rend bien des corps non vides* » — **le shell avait mangé mes antislashes**, donc je testais une regex différente de celle du fichier. C'est la lecture du **fichier lui-même** qui a tranché. *Un test qui n'emploie pas exactement le texte du fichier ne teste pas le fichier*.

6. L'état des preuves — temoins, instantane, mutations, passe

preuve	etat mesure
Instantane avant / apres	identique octet pour octet , sha256 cfcffd90645abe53 — verifie apres chacune des deux migrations, pas seulement a la fin
Sonde	25 cles (23 avant), les deux chemins conduits sur un jeu de cas commun — <i>c'est la comparaison qui fait la preuve</i>
Temoins (bloc CCCII)	16 , dont 7 qui figent ce que le hub NE fait pas
Mutations negatives	14 , toutes mordent sur leur propre temoin ; controle sain a 0 rouge avant ET apres
Passe parcours	3726 vert, 0 rouge, sur l'arbre FINAL — total predit = obtenu (3710 + 16)
Autres suites	calculs 339/339 · muscles 241/241 · croises 50/50 · dates 9/9 · donnees classees, 0 trou nouveau

Les mutations, et le temoin que chacune fait rougir

la mutation	rouges
« je finis le hub » : la pastille absorbee dedans	1 , exactement le temoin des 5 gestes omis
« le hub pose aussi la provenance » (il devient la douane)	2
« le hub valide » (douane prematuree)	1
le hub ecrit au journal (S. foodLog)	2
fusion par ressemblance : quickFillFood branchee	2
le champ des grammes n'est plus vide	2
le drapeau _bcQtyPose ne retombe plus	2
le hub touche un proprietaire de 1b/3	2
la 2 ^e porte reprend sa copie a la main	2
un geste propre a la 1 ^{re} porte absorbe par le hub	1
le libelle devient une formule unique	1 , exactement le temoin d'equivalence
PORTEE : une porte du groupe A cesse de passer par le chemin commun (6 -> 5)	1 , exactement le temoin de portee
PORTEE : readFoodLabelL redevient une porte (12 -> 13)	1 , exactement le temoin de portee

7. Verdict — le hub est-il pret pour la douane ?

OUI — LE HUB EST PRET POUR LA DOUANE

Le hub existe, deux chemins convergent, **rien de ce qui est enregistre n'a bouge** (instantane identique octet pour octet), et **ce qu'il refuse de faire est fige par 7 temoins** — c'est-a-dire que la douane aura un endroit ou se poser, et un filet qui dira si elle deborde.

Avec une reserve dite plutot que cachee : le hub a **2 appelants**, mais **6 portes sur 12** l'atteignent — les 5 du groupe A *indirectement* (via _offRemplirFormulaire, qui n'est pas lui-meme une porte), plus onFoodLabelFile en direct. Les 3 restantes du groupe B ne convergeront **que si on accepte de changer leur comportement** : c'est une **decision produit**, pas une extraction — et donc une question pour Michel, pas pour moi.

Ce qui reste ouvert, explicitement hors perimetre

La **douane** (consigne explicite : apres validation du hub seul) · les **3 portes du groupe B** non migrees · S. savedFoods multi-onglets · l'ecart **48,3 / 48** · l'historique · les migrations · les harmonisations produit · le garde !_bcNutr non bloquant, mesure et documente en ft-v1203, **en attente d'un feu vert separe**.

Conception complete : docs/HUB-NUTRITION.md. Ce PDF est genere par tools/gen_hub_pdf.py, dont les 21 gardes recomptent chaque chiffre depuis le code servi et **refusent de produire** si un seul fait tombe — y compris le total de la passe, qui est **lu dans son journal** et jamais ecrit a la main.